

Manual Técnico

Tabla de Contenido

Manual Técnico.....	1
1.1 Propósito	3
1.2 Alcance	3
1.3 Tecnologías Utilizadas	3
2. Arquitectura del Sistema	3
2.2 Descripción de Componentes	4
3. Diagrama de Clases	5
3.1 Descripción de Clases Principales	6
3.2 Enumeraciones (TokenType)	6
4. Autómata Finito Determinista (AFD)	7
4.2 Tabla de Transiciones	8
5. Algoritmos de Tokenización	9
5.1 Algoritmo Principal (nextToken).....	9
5.2 Reconocimiento de Fechas	9
5.4 Reconocimiento de Códigos ID.....	10
6. Recuperación de Errores	10
6.1 Estrategia de Recuperación	10
6.2 Tipos de Errores Detectados.....	11
7. Generación de Reportes	11
7.1 Reportes HTML Generados.....	11
7.2 Detección de Conflictos en Citas.....	12
7.3 Diagrama Graphviz	12
8. Justificación de Decisiones de Diseño	13
8.1 Por qué C++17	13
8.2 Por qué Qt para la GUI.....	13
8.3 Por qué implementación manual del AFD (sin std::regex).....	13

8.4 Por qué estructura de 4 reportes HTML.....	13
9. Estructura del Código Fuente.....	14
10. Casos de Prueba.....	15
11. Conclusiones	15
12. Referencias	16

1. Introducción

1.1 Propósito

MedLexer es una herramienta de análisis léxico y sintáctico diseñada para procesar archivos con extensión .med escritos en el lenguaje MedLang. El sistema permite tokenizar, validar y generar reportes estadísticos sobre datos médicos hospitalarios.

1.2 Alcance

Este manual describe la arquitectura, diseño e implementación del sistema, incluyendo:

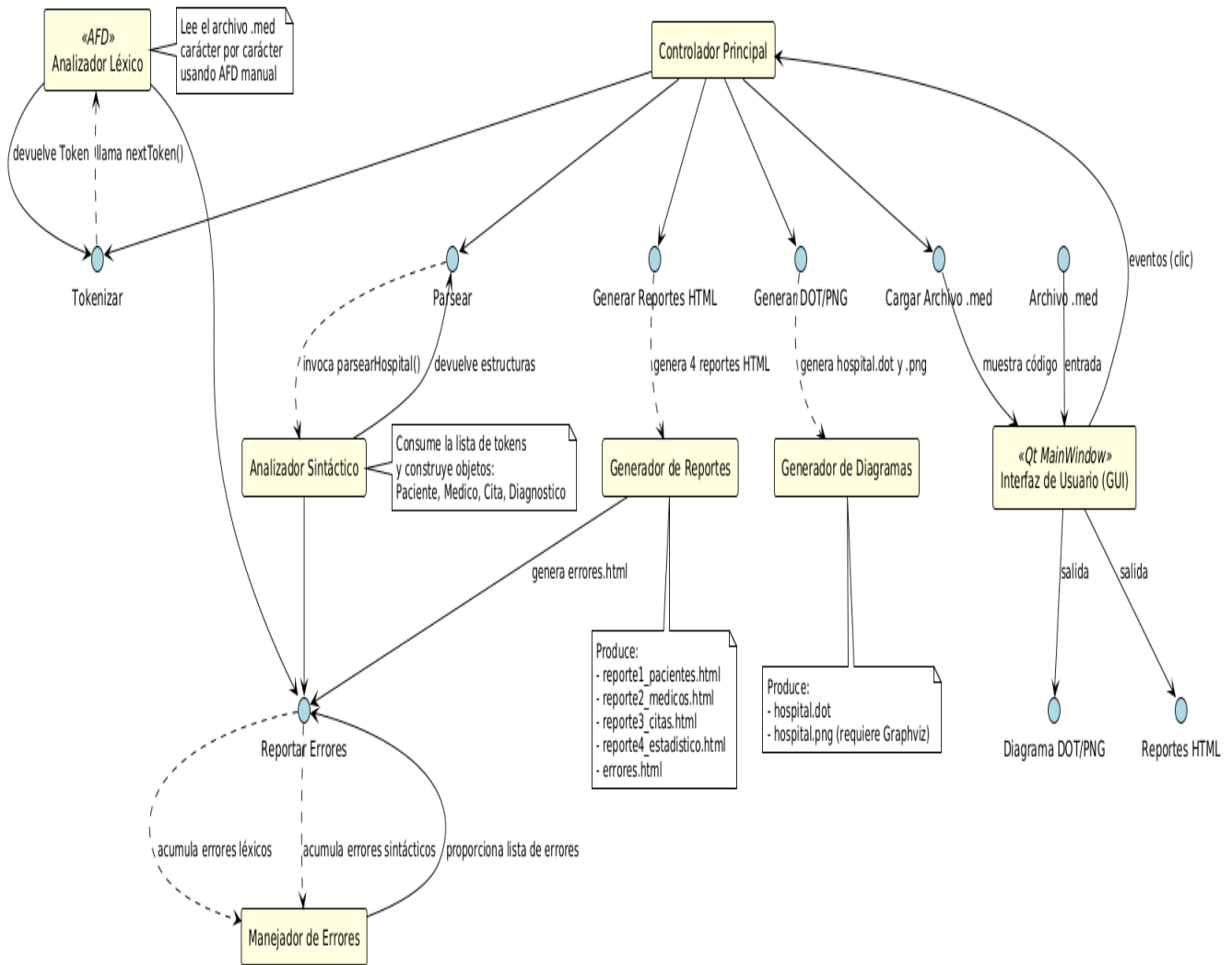
- Diagramas UML de componentes y clases
- Diagrama del Autómata Fiito Determinista (AFD)
- Tabla de transiciones del autómata
- Descripción de algoritmos de tokenización y recuperación de errores
- Justificación de decisiones de diseño

1.3 Tecnologías Utilizadas

Tecnología	Versión	Uso
C++	C++17	Lenguaje principal
Qt	6.x / 5.15	Interfaz gráfica
Graphviz	-	Visualización de diagramas
STL	C++17	Estructuras de datos

2. Arquitectura del Sistema

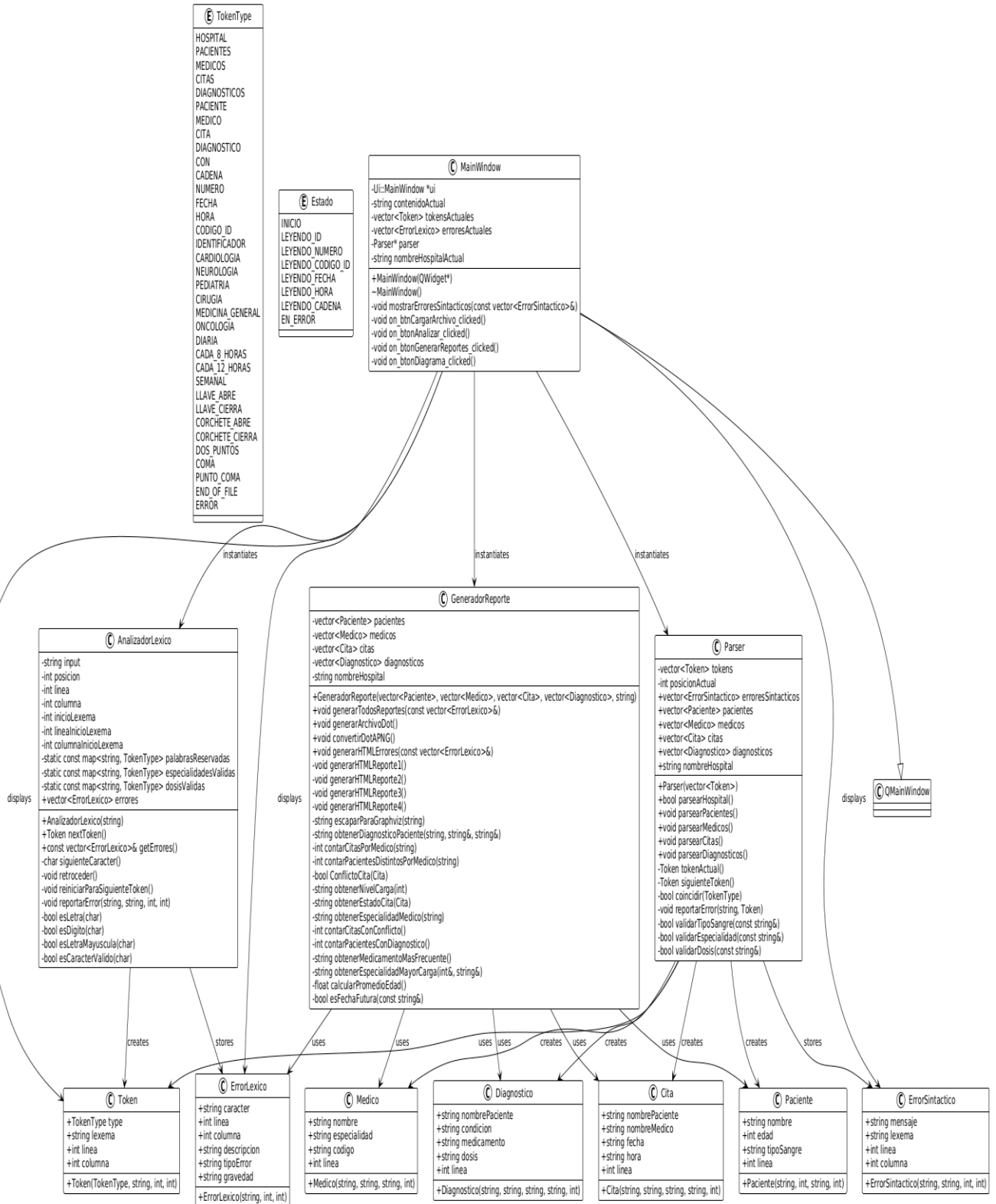
2.1 Diagrama de Componentes



2.2 Descripción de Componentes

Componente	Responsabilidad
Interfaz Gráfica (GUI)	Maneja la interacción con el usuario, carga de archivos, visualización de resultados
Analizador Léxico	Implementa el AFD para tokenización del código fuente
Parser	Realiza análisis sintáctico y construye estructuras de datos
Generador de Reportes	Crea archivos HTML con estadísticas y código DOT para Graphviz

3. Diagrama de Clases



3.1 Descripción de Clases Principales

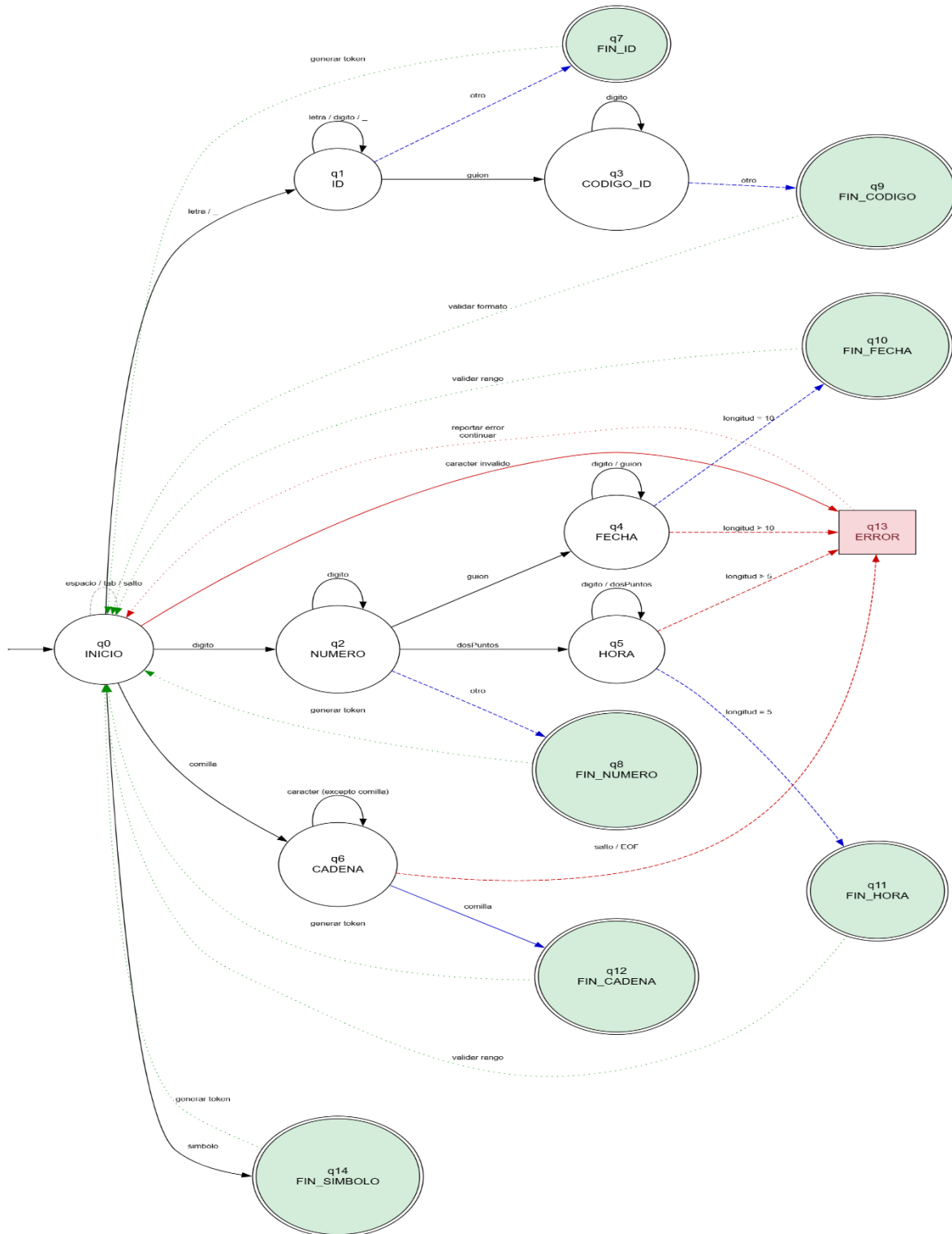
Clase	Atributos Principales	Métodos Principales
MainWindows	contenidoActual, tokensActuales, parser	cargarArchivo(), analizar(), generarReportes()
AnalizadorLexico	input, posicion, linea, columna, errores	nextToken(), getErrores()
Token	type, lexema, linea, columna	
Parser	tokens, posicionActual, pacientes, medicos, citas, diagnosticos	parsearHospital(), parsearPacientes(), parsearMedicos(), parsearCitas(), parsearDiagnosticos()
GeneradorReporte	pacientes, medicos, citas, diagnosticos	generarTodosReportes(), generarArchivoDot()

3.2 Enumeraciones (TokenType)

Categoría	Tokens
Palabras Reservadas	HOSPITAL, PACIENTES, MEDICOS, CITAS, DIAGNOSTICOS, PACINTE, MEDICO, CITA, DIAGNOSTICO, CON
Literales	CADENA, NUMERO, FECHA, HORA, CODIGO_ID, IDENTIFICADOR
Especialidades	CARDIOLOGIA, NEUROLOGIA, PEDIATRIA, CIRUGIA, MEDICINA_GENERAL, ONCOLOGIA
Dosis	DIARIA, CADA_8_HORAS, CADA_12_HORAS, SEMANAL
Símbolos	{}, [], :, ;

4. Autómata Finito Determinista (AFD)

4.1 Diagrama del AFD



4.2 Tabla de Transiciones

TABLA DE TRANSICIONES - AFD MEDLEXER			
Estado Actual	Entrada	Siguiente Estado	Acción
q0 (INICIO)	letra / _	q1 (LEYENDO_ID)	Avanzar
	dígito	q2 (LEYENDO_NUMERO)	Avanzar
	"	q6 (LEYENDO_CADENA)	Avanzar
	{ } [] : , ;	q14 (FIN_SIMBOLO)	Generar token
	espacio / tab / \n	q0 (INICIO)	Ignorar
	otro	q13 (ERROR)	Reportar error
q1 (LEYENDO_ID)	letra / dígito / _	q1 (LEYENDO_ID)	Avanzar
	-	q3 (LEYENDO_CODIGO_ID)	Avanzar
	otro	q7 (FIN_ID)	Generar token
q2 (LEYENDO_NUMERO)	dígito	q2 (LEYENDO_NUMERO)	Avanzar
	-	q4 (LEYENDO_FECHA)	Avanzar
	:	q5 (LEYENDO_HORA)	Avanzar
	otro	q8 (FIN_NUMERO)	Generar token
q3 (LEYENDO_CODIGO_ID)	dígito	q3 (LEYENDO_CODIGO_ID)	Avanzar
	otro	q9 (FIN_CODIGO_ID)	Validar formato
q4 (LEYENDO_FECHA)	dígito / -	q4 (LEYENDO_FECHA)	Avanzar
	longitud = 10	q10 (FIN_FECHA)	Validar formato
q5 (LEYENDO_HORA)	dígito / :	q5 (LEYENDO_HORA)	Avanzar
	longitud = 5	q11 (FIN_HORA)	Validar formato
q6 (LEYENDO_CADENA)	caracter excepto "	q6 (LEYENDO_CADENA)	Avanzar
	"	q12 (FIN_CADENA)	Generar token
	\n / EOF	q13 (ERROR)	Cadena sin cerrar
Nota: q7, q8, q9, q10, q11, q12, q14 son estados de aceptación (generan token)			

5. Algoritmos de Tokenización

5.1 Algoritmo Principal (nextToken)

1. Mientras haya espacios en blanco, avanzar y actualizar línea/columna
2. Si es fin de archivo, retornar EOF
3. Guardar posición de inicio del token
4. Según el primer carácter, entrar en estado correspondiente:
 - Letra → LEYENDO_ID
 - Dígito → LEYENDO_NUMERO
 - " → LEYENDO_CADENA
 - Símbolo → retornar token de símbolo
5. Procesar caracteres según estado actual:
 - En cada paso, leer siguiente carácter
 - Actualizar estado según transiciones
6. Cuando se alcanza estado de aceptación:
 - Extraer lexema desde inicio hasta posición actual
 - Determinar tipo de token
 - Retornar Token(lexema, tipo, línea, columna)

5.2 Reconocimiento de Fechas

Formato esperado: AAAA-MM-DD

Validaciones:

- Año: 4 dígitos (ejemplo: 2025)

- Mes: 01-12
- Día: 01-31 (validado según mes, incluyendo años bisiestos)
- Hola

5.4 Reconocimiento de Códigos ID

Formato esperado: LLL-DDD

Validaciones:

- 3 letras mayúsculas o minúsculas
- Guion
- 1 o más dígitos

Ejemplos válidos: MED-001, CAR-123, PED-789

6. Recuperación de Errores

6.1 Estrategia de Recuperación

El sistema implementa una estrategia de recuperación de errores que permite continuar el análisis después de encontrar un error léxico o sintáctico.

Pasos de recuperación:

1. Detectar el error y reportarlo con línea y columna exacta
2. Consumir el token inválido
3. Sincronizar al siguiente punto válido:
 - Para errores léxicos: continuar con el siguiente carácter

- Para errores sintácticos: avanzar hasta encontrar una palabra clave conocida (PACIENTES, MEDICS, CITAS, DIAGNOSTICOS) o un delimitador de estructura ({, }, [,], ,, ;)

6.2 Tipos de Errores Detectados

Tipo de Error	Ejemplo	Mensaje
Caracter inválido	@	"Caracter no permitido en el lenguaje"
Cadena sin cerrar	"Hola	"Cadena sin cerrar. Falta la comilla de cierre"
Fecha inválida	2025-13-10	"Fecha inválida: mes fuera de rango"
Hora inválida	25:00	"Hora inválida. Use rango 00:00 - 23:59"
Código ID inválido	MED-00X	"Formato de código inválido"
Especialidad inválida	Cardio	"Especialidad médica no válida"
Dosis inválida	CADA_24_HORAS	"Dosis no válida. Use: DIARIA, CADA_8_HORAS..."
Tipo sangre inválido	"O++"	"Tipo de sangre inválido"

7. Generación de Reportes

7.1 Reportes HTML Generados

Reporte	Archivo	Contenido
Reporte 1	reporte1_pacientes.html	Historial de pacientes con diagnósticos activos
Reporte 2	reporte2_medicos.html	Carga de médicos por especialidad

Reporte	Archivo	Contenido
Reporte 3	reporte3_citas.html	Agenda de citas con detección de conflictos
Reporte 4	reporte4_estadistico.html	Estadísticas generales del hospital

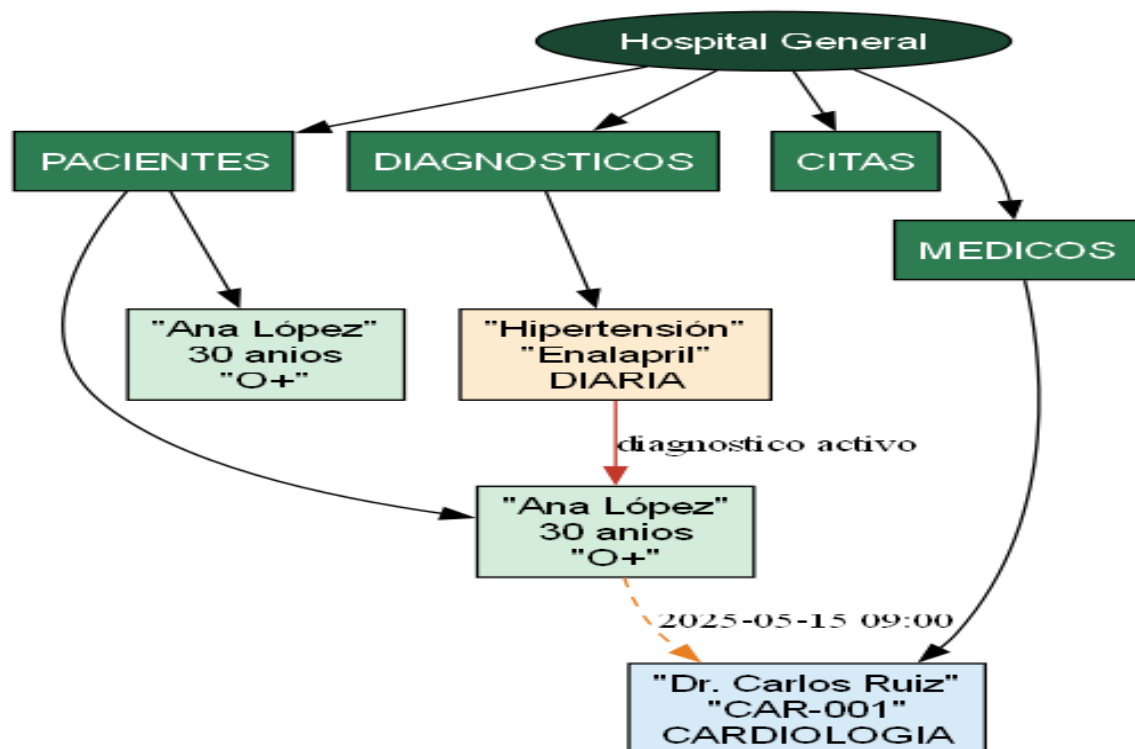
7.2 Detección de Conflictos en Citas

El sistema detecta conflictos de horario cuando:

- Mismo médico
- Misma fecha
- Misma hora

7.3 Diagrama Graphviz

El sistema genera un archivo hospital.dot que representa la jerarquía del hospital:



8. Justificación de Decisiones de Diseño

8.1 Por qué C++17

- **Rendimiento:** El análisis léxico requiere procesamiento eficiente de grandes volúmenes de datos
- **Control de memoria:** Permite manejo manual de memoria para optimizar el análisis
- **Portabilidad:** Código compilable en Windows, Linux y macOS
- **STL:** Estructuras de datos eficientes (vector, map, string)

8.2 Por qué Qt para la GUI

- **Multiplataforma:** Funciona en Windows, Linux y macOS
- **Documentación extensa:** Facilita el desarrollo
- **Componentes ricos:** QWidget, QTextEdit, QFileDialog listos para usar
- **Comunidad activa:** Soporte y ejemplos disponibles

8.3 Por qué implementación manual del AFD (sin std::regex)

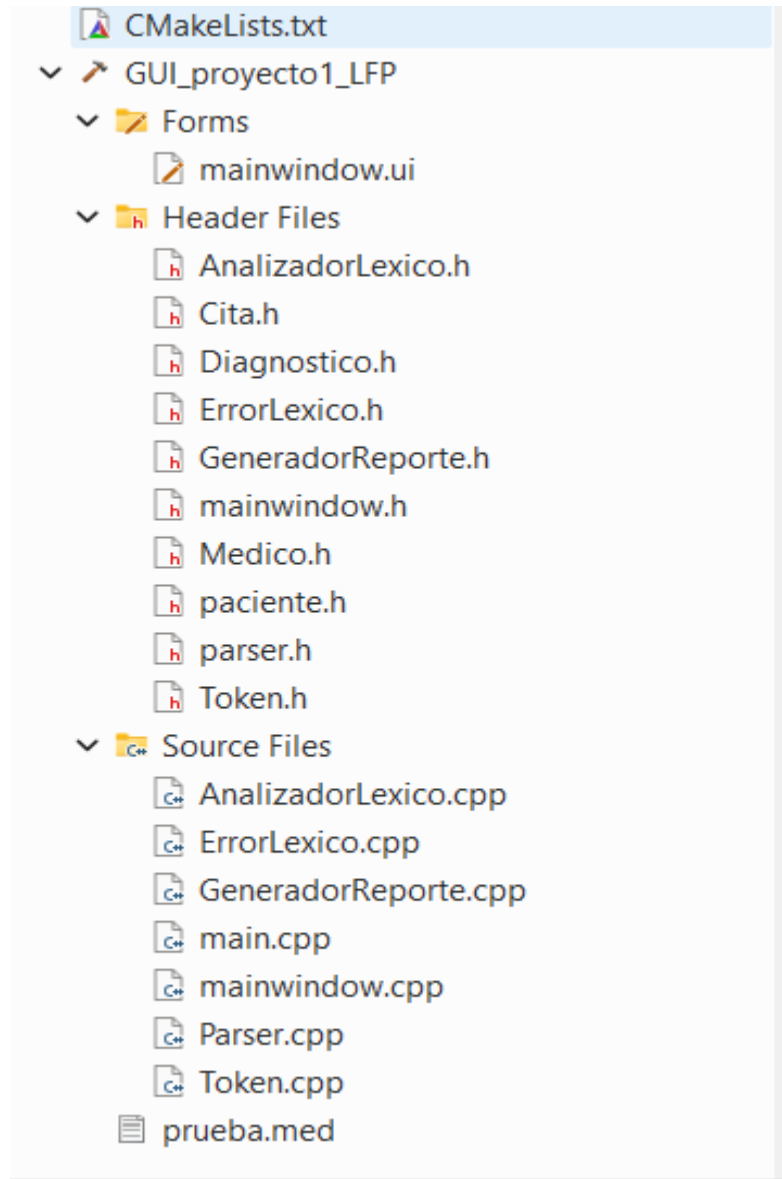
- **Propósito educativo:** Comprender en profundidad la teoría de autómatas
- **Control preciso:** Mayor control sobre el proceso de tokenización
- **Rendimiento:** std::regex puede ser más lento para análisis léxico
- **Manejo de errores:** Mayor flexibilidad para reportar errores con posición exacta
- **Requisito del proyecto:** Especificado en el enunciado

8.4 Por qué estructura de 4 reportes HTML

- **Visualización clara:** HTML permite presentación estructurada con CSS

- **Portabilidad:** Los reportes pueden abrirse en cualquier navegador
- **Facilidad de implementación:** Generación directa desde C++ con strings
- **Requerimiento del proyecto:** Según la sección 3.4 del enunciado

9. Estructura del Código Fuente



10. Casos de Prueba

Dentro de este documento PDF se encuentra la documentación de los casos de prueba hechos en el programa:



casos de prueba.pdf

11. Conclusiones

MedLexer implementa exitosamente un analizador léxico y sintáctico para el lenguaje MedLang, cumpliendo con los siguientes objetivos:

1. Tokenización precisa de todos los tipos de tokens definidos
2. Detección de errores léxicos con posición exacta (línea y columna)
3. Recuperación de errores que permite continuar el análisis
4. Generación de 4 reportes HTML con estadísticas médicas
5. Generación de diagrama Graphviz con la jerarquía del hospital
6. Interfaz gráfica intuitiva en Qt

12. Referencias

- Aho, A. V., Sethi, R., Ullman, J. D., & Lam, M. S. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- Qt Documentation. (2026). Qt 6.0 Reference. <https://doc.qt.io/qt-6/>
- Graphviz Documentation. (2026). DOT Language. <https://graphviz.org/doc/info/lang.html>

Autor: Jonathan Eduardo Fuentes García

Carnet: 202408977